

Types

DM580: Functional Programming

Casper Bach

Reading Material

This lecture relates to these chapters of *Programming in Haskell, 2nd ed.*

- Chapter 3: Types and Classes

A large, dark gray, stylized lambda symbol (λ) is positioned on the left side of the image, partially overlapping the text.

First, a Service Announcement

Pass/Fail Assignments

- **Voluntary** to submit
- But **strongly recommended**: covers basic material you are expected to master
- You will receive feedback on assignments only if they are **handed in before the deadline**
- **Office hours**: Tuesdays from 14:30-16:15. My office: Ø13-604a-1
 - Drop by for questions about material or course in general

Course Plan and Reading

Week 39 Introduction and Haskell Preliminaries <i>Read:</i> * Foreword + 1 + 2	Week 40 Types and Basic Functions <i>Read:</i> 3 + 4	Week 41 Lists and Recursion <i>Read:</i> 5 + 6 <i>Submit handin assn. 1</i>	Week 42 <i>Fall Break</i>
Week 43 Higher-Order Functions and Type Classes <i>Read:</i> 7 + 8 <i>Submit handin assn. 2</i>	Week 44 Programming with I/O <i>Read:</i> 10 <i>Submit handin assn. 3</i>	Week 45 Monads <i>Read:</i> 12	Week 46 <i>No lecture</i> <i>Read:</i> TBA <i>Submit handin assn. 4</i>
Week 47 Concurrency <i>Read:</i> Beautiful Concurrency	Week 48 Lazy Evaluation <i>Read:</i> 15 <i>Submit handin assn. 5</i>	Week 49 Exam Project <i>Read:</i> TBA	Week 50 Q&A Session

* Except where otherwise indicated, reading refers to Graham Hutton's *Programming in Haskell*, second edition



What are Types,
and Why do They Matter?

What is a Type?

What is a Type?

- A collection of values, often associated with primitive operations well-defined for such values

What is a Type?

- A collection of values, often associated with primitive operations well-defined for such values
- How do compilers and language runtimes utilize types?

What is a Type?

- A collection of values, often associated with primitive operations well-defined for such values
- How do compilers and language runtimes utilize types?
 - Dynamically. Checking at runtime that we do not accidentally call primitive operations with wrong parameters

What is a Type?

- A collection of values, often associated with primitive operations well-defined for such values
- How do compilers and language runtimes utilize types?
 - Dynamically. Checking at runtime that we do not accidentally call primitive operations with wrong parameters
 - Statically. Checking at compile time that we cannot accidentally call primitive operations with wrong parameters

Why Type Check?

Because type checking ...

- **Eliminates entire classes of bugs:** type errors
- **Catches bugs at compile time**, which is typically the cheapest point in time
- **Enforces and documents invariants** that would be laborious to test for
- **Supports developer tooling:** auto-completion, safe renaming, jump to definition, etc. leverage the static typing structure of programs
- **Lets us refactor with confidence:** change a type, and the type checker will notify you of all* the places in your code that need change

* Depends on how strongly typed your program is

Learning Objectives – Types

- Explain common types in Haskell
- Write simple functions and type signatures in Haskell
- Explain what an overloaded type is

Types and Safety

Types and Safety

- Programs are only as safe as their types allow them to be

Types and Safety

- Programs are only as safe as their types allow them to be
- Some invariants can be difficult or downright impossible to encode (depending on the type system)
 - Depends on the expressiveness of the underlying type system
 - Fine-grained invariants can be expressed with *dependent types* (beyond the scope of this course) require functions that resemble (and *are*) proofs¹

¹See, e.g., elective courses on *Programming Language Foundations* or *Advanced Functional Programming*

Types and Safety

“Stringly-typed” vs. typed programming

- Goal: write a function that ships an order only if the order is paid
- Try these in GHCi. Which function is safer and why?

```
ship1 :: String → String
ship1 "paid" = "shipped"
ship1 _      = "error"
```

```
data Order = Pending | Paid | Shipped
```

```
ship2 :: Order → Maybe Order
ship2 Paid = Just Shipped
ship2 _    = Nothing
```

Types and Safety

Typed programming vs. dependently typed programming

- Goal: write a function that takes the head of a list
- Try the left in GHCi. Which is safer and why?

```
data N : Set where
  zero : N
  suc   : N → N
data Vec (A : Set) : N → Set where
  []      : Vec A zero
  _::_    : ∀ {n} → A → Vec A n → Vec A (suc n)
```

```
head :: [a] → a
head (x : []) = x
```

Haskell

```
head : ∀ {A n} → Vec A (suc n) → A
head (x :: xs) = x
```

Agda

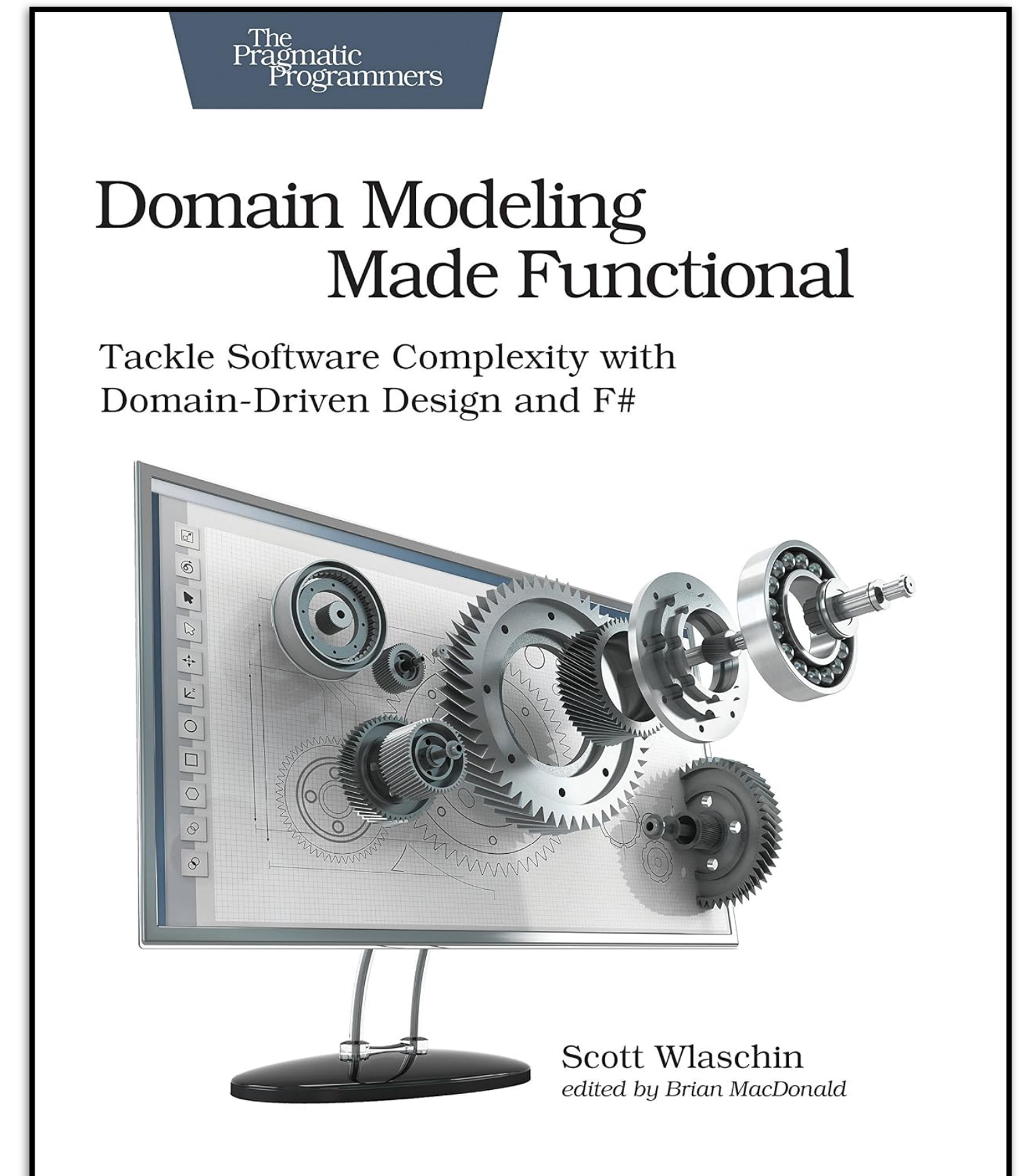
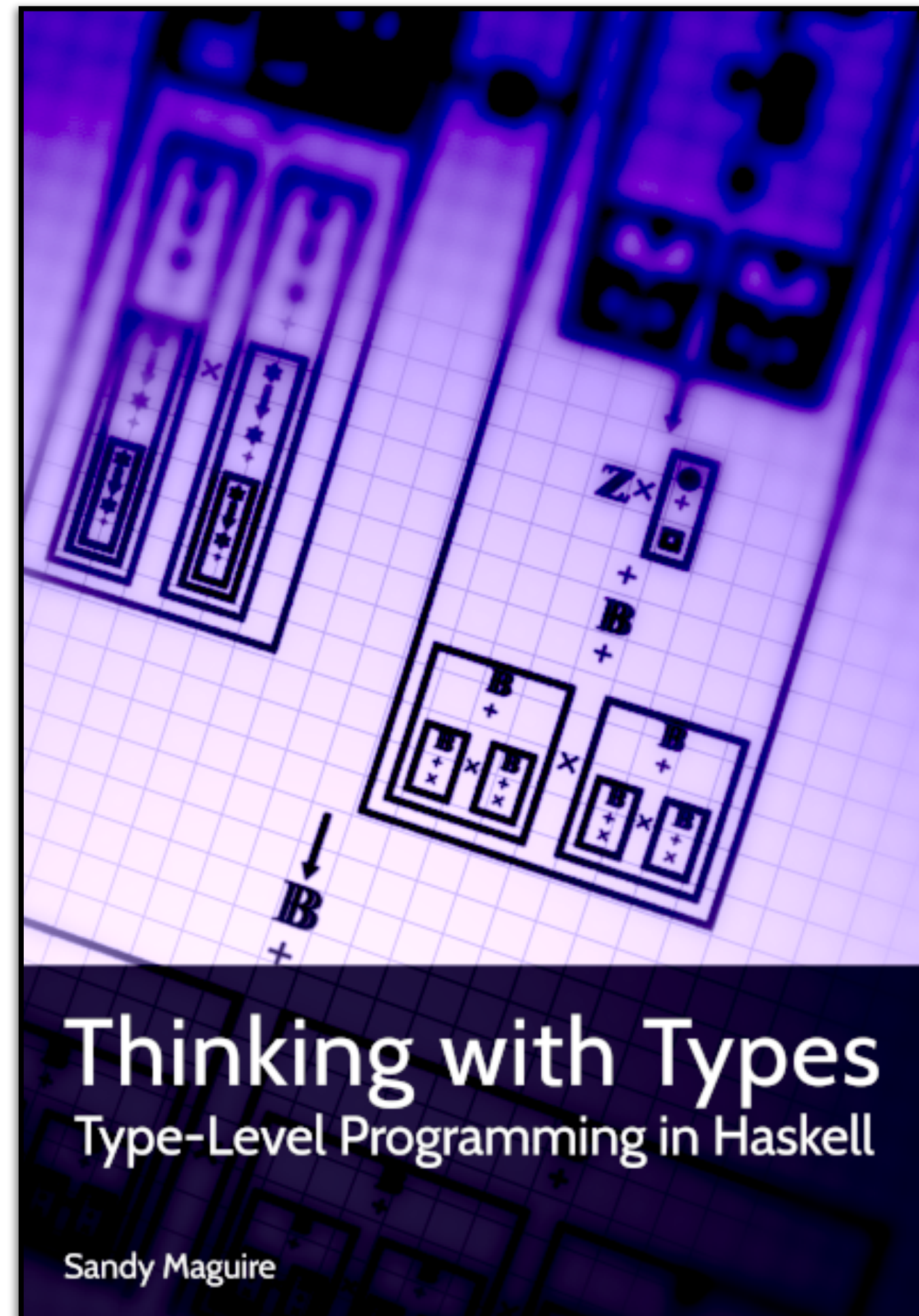
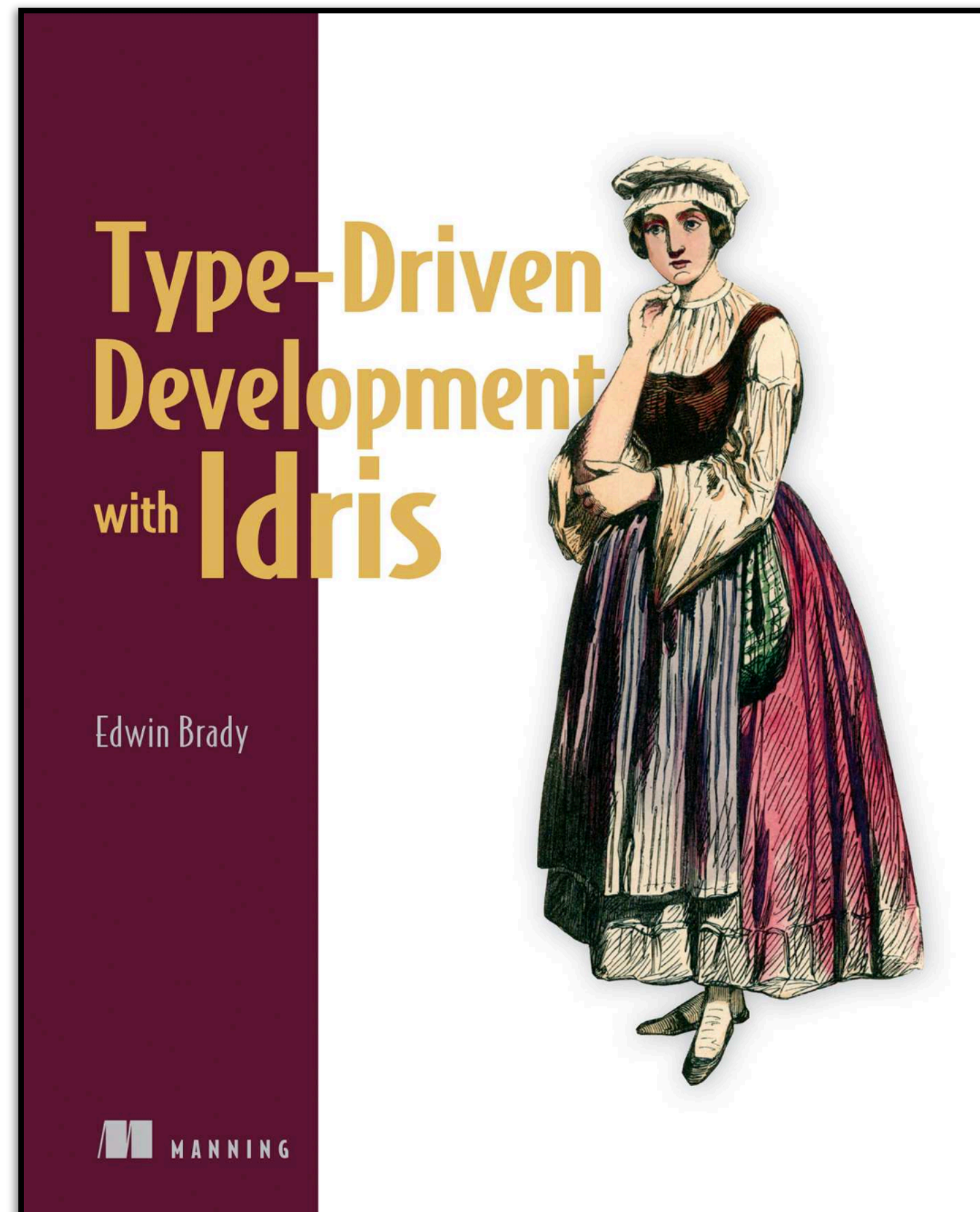
Thinking With Types

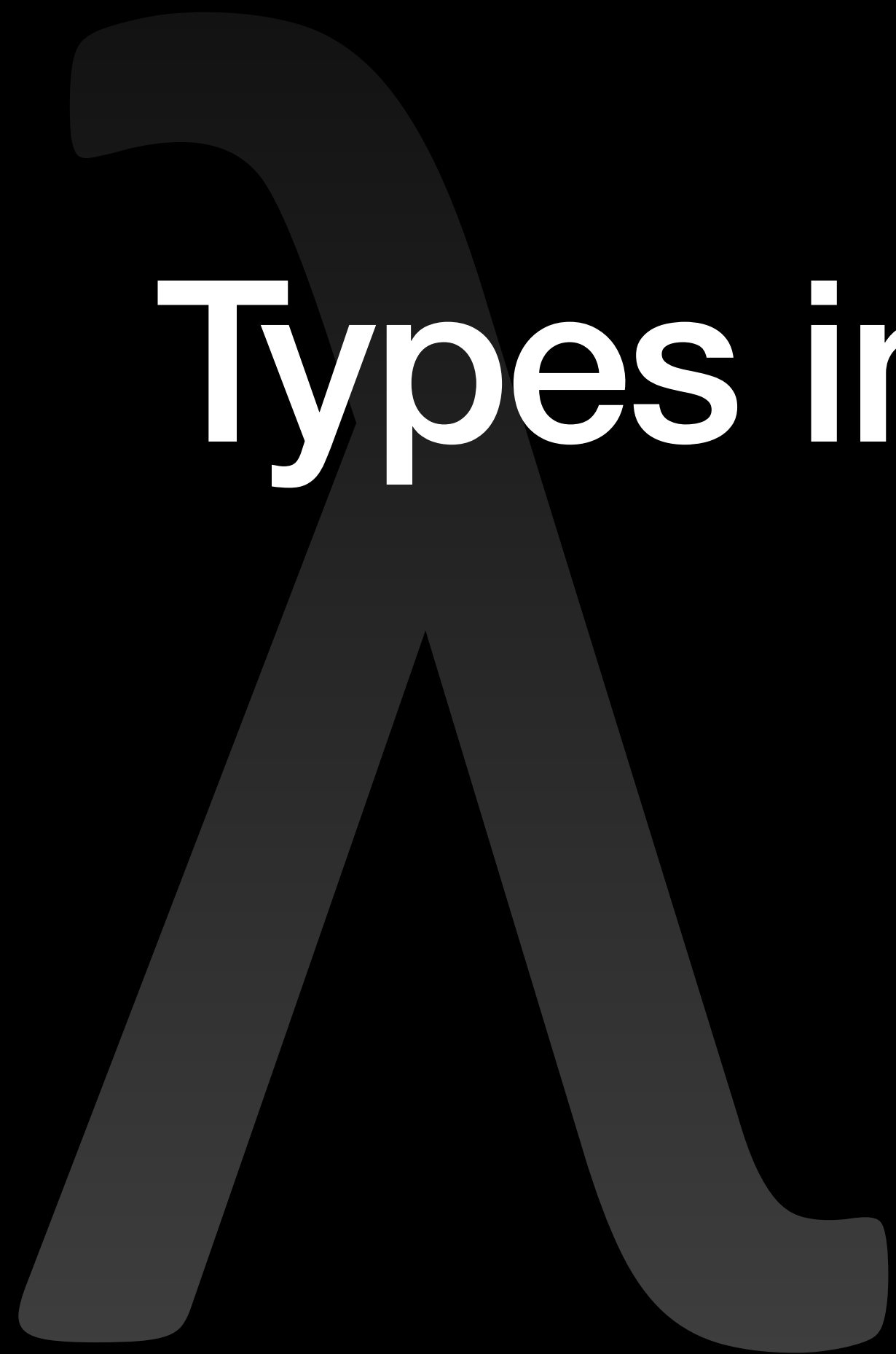
Guiding principles

- Design types to model only possible values
- Model domain so that illegal states are impossible to represent
- Use type signatures to convey invariants and obligations the compiler can check for you so you don't have to
- An art that takes time to master
- Hutton's "Programming in Haskell" contains examples that we will cover

Thinking With Types

Entire books are written on the subject





Types in Haskell

Example Type: Integers

Values

- $\{0, 1, -1, 2, -2, \dots\}$

Operations

- $_+_ : \text{Int} \times \text{Int} \rightarrow \text{Int}$
- $_-_ : \text{Int} \times \text{Int} \rightarrow \text{Int}$
- $\dots$

Example Type: Integers in Haskell

Values

- $\{0, 1, -1, 2, -2, \dots\}$

Functions

- $(+) :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
- $(-) :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
- (**Caveat:** $(+)$ in Haskell is actually an *overloaded* type; we will return to this point later.)
- ...

Example Type: Booleans in Haskell

Exercise

Values

- ?

Functions

- ?

Example Type: Lists in Haskell

Values

- `[1, 2, 3]`
- `[]`
- `1 : 2 : 3 : []`
- `[1] : [2] : []`
- ...

Functions

- `head :: [a] → a`
- `tail :: [a] → [a]`
- `(++) :: [a] → [a] → [a]`
- ...

Example Type: Maybe in Haskell

Values

- `data Maybe a`
 `= Nothing`
 `| Just a`
- `Just 0 :: Maybe Int`
- `Nothing :: Maybe Bool`
- `Just (Just True)`
 `:: ?`

Functions

- `maybe :: b → (a → b)`
 `→ Maybe a → b`
- `case v of`
 `Just x → ...`
 `Nothing → ...`
- `foo :: Maybe Int → Int`
 `foo (Just x) = x`
 `foo Nothing = 0`
- We can case split and pattern match on any declared data type in Haskell

... Let's look at examples using GHCi

A large, dark gray, stylized lambda symbol (λ) is positioned on the left side of the image, partially overlapping the text.

Overloaded Types

The Same Operator Can Have Multiple Implementations

The Same Operator Can Have Multiple Implementations

- For integers:

$$\begin{array}{c} > 1 + 1 \\ 2 \end{array}$$

The Same Operator Can Have Multiple Implementations

- For integers:

$$\begin{array}{l} > 1 + 1 \\ 2 \end{array}$$

- For floating-point numbers:

$$\begin{array}{l} > 1.3 + 2.9 \\ 4.2 \end{array}$$

The Same Operator Can Have Multiple Implementations

- For integers:

$$\begin{array}{r} > 1 + 1 \\ 2 \end{array}$$

- For floating-point numbers:

$$\begin{array}{r} > 1.3 + 2.9 \\ 4.2 \end{array}$$

- What is (+) really, then?

Numbers in Haskell, Revisited

Values

- $\{0, 1, -1, 2, -2, \dots\}$

Functions

- $(+) : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
- $(-) : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
- (**Caveat:** $(+)$ in Haskell is actually an *overloaded type*; we will return to this point later.)
- ...

Numbers in Haskell, Revisited

Values

- $\{0, 1, -1, 2, -2, \dots\}$

Functions

- $(+)$: $\text{Num } a \Rightarrow a \rightarrow a \rightarrow a$
- $(-)$: $\text{Num } a \Rightarrow a \rightarrow a \rightarrow a$
- ...

Overloaded Types: Numbers

- An overloaded type is programmed against an *interface*
- Consider $(+) :: \text{Num } a \Rightarrow a \rightarrow a \rightarrow a$
 - $\text{Num } a$ is the interface of the type a
 - The function is *generic* in what the type a is: It is any type for which there exists an implementation of all functions of the interface Num
 - When we apply $(+)$, Haskell's type checker will automatically figure out which implementation to use, based on the type $(+)$ is applied to
- In Haskell, such interfaces are called *type classes*

The Num Type Class

```
> :info Num
type Num :: * → Constraint
class Num a where
    (+) :: a → a → a
    (-) :: a → a → a
    (*) :: a → a → a
    negate :: a → a
    abs :: a → a
    signum :: a → a
    fromInteger :: Integer → a
-- [truncated]
```

The Num Type Class

```
> :info Num
-- [continued]
{-# MINIMAL (+), (*), abs, signum, fromInteger, (negate | (-)) #-}
      -- Defined in 'ghc-internal-9.1003.0:GHC.Internal.Num'
instance Num Double
      -- Defined in 'ghc-internal-9.1003.0:GHC.Internal.Float'
instance Num Float
      -- Defined in 'ghc-internal-9.1003.0:GHC.Internal.Float'
instance Num Int
      -- Defined in 'ghc-internal-9.1003.0:GHC.Internal.Num'
instance Num Integer
      -- Defined in 'ghc-internal-9.1003.0:GHC.Internal.Num'
instance Num Word
      -- Defined in 'ghc-internal-9.1003.0:GHC.Internal.Num'
```

The Num Type Class

The Num Type Class

$\> :t \ (+)$
 $(+) :: \text{Num } a \Rightarrow a \rightarrow a \rightarrow a$

The Num Type Class

$> :t (+)$
 $(+) :: Num\ a \Rightarrow a \rightarrow a \rightarrow a$

$> :t (*)$
 $(*) :: Num\ a \Rightarrow a \rightarrow a \rightarrow a$

The Read Type Class

The Read Type Class

```
> :t read  
read :: Read a  $\Rightarrow$  String  $\rightarrow$  a
```

The Read Type Class

```
> :t read  
read :: Read a  $\Rightarrow$  String  $\rightarrow$  a  
  
> read "False"
```

The Read Type Class

```
> :t read  
read :: Read a  $\Rightarrow$  String  $\rightarrow$  a
```

```
> read "False"  
*** Exception: Prelude.read: no parse
```

The Read Type Class

```
> :t read  
read :: Read a  $\Rightarrow$  String  $\rightarrow$  a
```

```
> read "False"  
*** Exception: Prelude.read: no parse
```

```
> read "False" :: Bool
```

The Read Type Class

```
> :t read  
read :: Read a => String -> a
```

```
> read "False"  
*** Exception: Prelude.read: no parse
```

```
> read "False" :: Bool
```



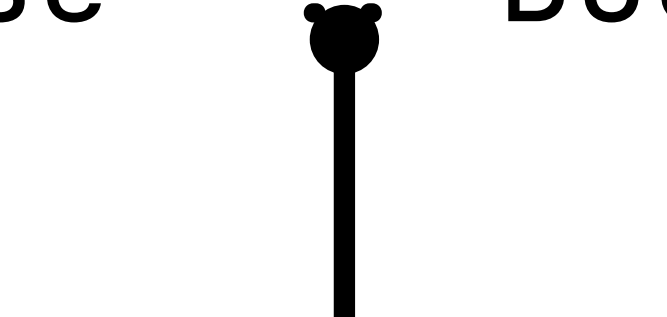
type annotation

The Read Type Class

```
> :t read  
read :: Read a => String -> a
```

```
> read "False"  
*** Exception: Prelude.read: no parse
```

```
> read "False" :: Bool  
False
```



type annotation

Examples of Type Classes

- `class Eq a where`
 - `(=) :: a → a → Bool`
 - `(/=) :: a → a → Bool`
- `class Show a where`
 - `show :: a → String`
- More examples can be found in the book Chapter 3 and in Haskell's Prelude

Exercise 1

- What type could we assign to the following function, and why?

```
wholeNumDiv x y = if x `mod` y == 0  
                  then Just (x `div` y)  
                  else Nothing
```

Exercise 2

- How would you implement the following functions?
- Why are they “safer” than the traditional `head` and `tail` functions?

`safeHead` $:: [a] \rightarrow \text{Maybe } a$

`safeTail` $:: [a] \rightarrow \text{Maybe } [a]$

A large, dark gray, stylized lambda symbol (λ) is positioned on the left side of the image, extending from the top to the bottom. It has a thick, rounded stroke.

Questions?